

Software y estándares para la Web

P3. XML : ESQUEMAS Y PROCESAMIENTO

Contenido

Objetivos	2
Temática del proyecto: “LaVuelta2026”	2
Ejercicio 1: Uso de XML Esquemas para validar los archivos XML de la etapa	3
Tarea 1.1: Generación del archivo etapa.xsd a partir del archivo etapa.dtd.....	3
Guía de la solución de la tarea 1.1	3
Tarea 1.2: Modificar el archivo etapa.xsd para especificar archivos XML usando las características de los XML esquemas.....	3
Guía de la solución de la tarea 1.2	3
Tarea 1.3: Crear un archivo etapaEsquema.xml a partir del archivo etapa.xml.....	4
Guía de la solución de la tarea 1.3	4
Tarea 1.4: Validar el archivo etapaEsquema.xml con etapa.xsd.....	4
Guía de la solución de la tarea 1.4	4
Tarea 1.5: Generar gráficamente el árbol DOM del archivo etapa.xsd denominado “esquemaEtapa.svg”	4
Guía de la solución de la tarea 1.5	4
Ejercicio 2: Generar archivos KML a partir del archivo etapaEsquema.xml usando el lenguaje Python	4
Guía de la solución del ejercicio 2	5
Ejercicio 3: Generar el archivo de altimetría de la etapa en formato SVG a partir del archivo etapaEsquema.xml usando el lenguaje Python	7
Guía de la solución del ejercicio 3	8
Resultados	9

Objetivos

En esta práctica se va a realizar:

- El uso de los estándares KML y SVG (Teoría “Introducción a XML”)
- La validación del documento XML con un XML Schema (Teoría “XML Schemas”)
- El lenguaje XPath (Teoría “XPath”)
- El procesamiento del documento XML y la generación de nuevos documentos XML (Teoría “Procesamiento y generación de XML”)

Temática del proyecto: “LaVuelta2026”

Se continúa con el proyecto “LaVuelta2026”. Este proyecto es evolutivo y será ampliado, completado y modificado en las diferentes prácticas de la asignatura.

Ejercicio 1: Uso de XML Esquemas para validar los archivos XML de la etapa

Se tienen que usar XML esquemas para validar los archivos de la etapa

Tarea 1.1: Generación del archivo **etapa.xsd** a partir del archivo **etapa.dtd**

Utilizando una herramienta se genera una primera versión del archivo XML esquema denominado **etapa.xsd** a partir del archivo **etapa.dtd** construido en la práctica anterior.

Guía de la solución de la tarea 1.1

Se aconseja utilizar como herramienta “**Visual Studio Community Edition**”, explicado en el tema de teoría “XML Schemas”.

<https://visualstudio.microsoft.com/es/vs/community/>

Se abre el archivo **etapa.dtd** con la herramienta, aparece un **menú denominado XML** y se elige la opción “**Esquemas ...**” para genera el archivo **etapa.xsd**

El problema es que el archivo generado **etapa.xsd** tiene las mismas limitaciones de los DTD y ninguna de las ventajas de los XML esquema.

Tarea 1.2: Modificar el archivo **etapa.xsd** para especificar archivos XML usando las características de los XML esquemas

Se solicita modificar el archivo **etapa.xsd** usando las características de los XML esquemas. Se deben utilizar tipos de datos, rangos y restricciones. También se debe revisar la codificación. Se generará un archivo denominado **informeXMLesquema.pdf** donde se deben explicar los cambios realizados en **etapa.xsd**. Este informe tendrá al menos los siguientes apartados: Cambios de codificación, cambios de tipos de datos, cambios de rangos y restricciones introducidas.

Guía de la solución de la tarea 1.2

Se debe editar el archivo **etapa.xsd** obtenido en la tarea anterior y modificarlo para que use las características propias de los XML esquema.

Los DTD solamente tienen un tipo de datos “**string**” denominado PCDATA (en elementos) y CDATA (en atributos). Sin embargo, XML esquema tiene una amplia variedad de tipos de datos que permiten especificar mucho mejor los archivos XML.

Deben usarse tipos de datos para especificar valores enteros, float, fecha, hora, duración, etc...

Los DTD solamente especifican rangos con “?”, “+”, “*”. Sin embargo, XML esquema permite también especificar rangos entre dos valores numéricos. También se deben poner restricciones al rango que pueden tomar los valores.

La herramienta también nos puede cambiar la codificación, asegurar que se sigue en “UTF-8”.

Los elementos deben tener el prefijo “**http://www.uniovi.es**”

Tarea 1.3: Crear un archivo **etapaEsquema.xml** a partir del archivo **etapa.xml**

Se crea un archivo **etapaEsquema.xml** que contiene la información de la etapa, pero con un enlace al archivo XML esquema denominado **etapa.xsd**, utilizando como prefijo de los elementos **"http://www.uniovi.es"**.

Se partirá del archivo **etapa.xml** de la práctica anterior.

Guía de la solución de la tarea 1.3

El archivo **etapaEsquema.xml** es el archivo **etapa.xml** al que se ha eliminado la línea que contiene el enlace al archivo **etapa.dtd** y debe modificarse por el enlace al archivo **etapa.xsd**

Consultar los ejercicios resueltos del tema de teoría "XML Schemas" en los cuales se introduce el enlace a los XML esquemas con el prefijo **"<http://www.uniovi.es>"**.

Tarea 1.4: Validar el archivo **etapaEsquema.xml** con **etapa.xsd**

Se debe validar el archivo "etapaEsquema.xml" con el archivo XML esquema "etapa.xsd"

Guía de la solución de la tarea 1.4

Utilizar una herramienta o un editor con validación de XML esquemas para comprobar que el archivo **etapaEsquema.xml** es válido con **etapa.xsd**

Se aconseja usar "Visual Studio Code" con el plug-in "XML language tools red hat".

Tarea 1.5: Generar gráficamente el árbol DOM del archivo **etapa.xsd** denominado **"esquemaEtapa.svg"**

El archivo XML esquema denominado **etapa.xsd** es un archivo XML y por tanto se puede visualizar su árbol DOM con un archivo denominado **"esquemaEtapa.svg"** generado con la herramienta "xml2svg.exe"

Se debe generar el archivo "esquemaEtapa.svg", también se debe generar el archivo **"esquemaEtapaq.pdf"**

Guía de la solución de la tarea 1.5

Usar la herramienta "xml2svg.exe" para generar el archivo "esquemaEtapa.svg" desde una terminal en línea de comandos. Usar el navegador Opera para generar el PDF.

Ejercicio 2: Generar archivos **KML** a partir del archivo **etapaEsquema.xml** usando el lenguaje Python

Se deben generar un archivo en formato KML denominados **"etapa.kml"** a partir del archivo **"etapaEsquema.xml"**. Debe usarse obligatoriamente el lenguaje **Python**, se debe crear un archivo denominado **"xml2kml.py"**.

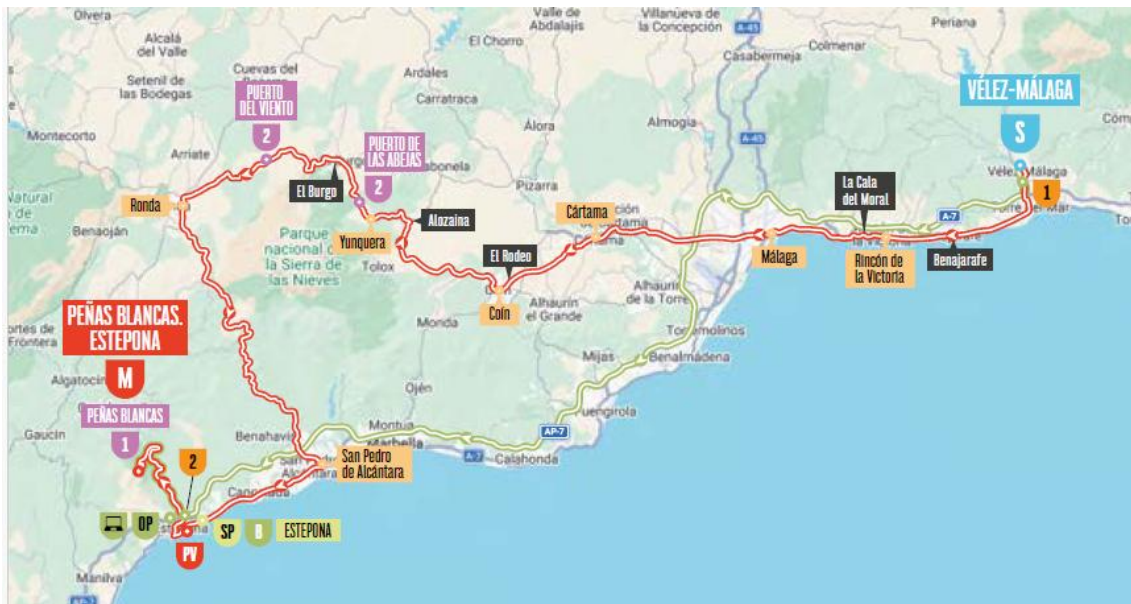
Es obligatorio utilizar “**expresiones XPath**” para obtener la información del archivo XML. También es obligatorio que el programa sea totalmente orientado a objetos (todas las funciones deben ser métodos de una clase).

Los archivos KML representan la **planimetría** de la etapa y pueden visualizarse con distintas herramientas, se debe capturar la imagen de la planimetría de la etapa en un archivo en formato PDF denominados “**planimetría.pdf**”. El archivo KML se mostrará sobre la imagen fotográfica de la etapa.

En el archivo KML se marcarán en rojo los puntos de salida y meta de la etapa, en verde los puertos de montaña, en azul los sprint intermedios y en amarillo los puntos anónimos.

Dado que las etapas pueden ser muy sinuosas se limita a **20 o 30 puntos** los puntos anónimos.

A continuación, se muestra la planimetría de la etapa 19 obtenida del libro de ruta oficial de “La Vuelta 2026”. Se debe obtener una **aproximación** a la planimetría presentada en el libro de ruta.



Guía de la solución del ejercicio 2

El formato de los archivos KML se explica en el *tema 1: Introducción a XML*. Este formato puede representar distintos elementos gráficos.

Cómo ejemplo de representación de **un punto** puede verse el ejemplo **Oviedo.kml** en el *tema 1: Introducción a XML*.

También se pueden representar **líneas**, puede verse en el ejemplo **N1612080.kml** en el *tema 6: Procesamiento y generación de XML* (06-Ejemplos\Nikon).

Además, puede verse una **combinación de puntos y líneas** en el ejemplo **rutaOviedo.kml** en el *tema 6: Procesamiento y generación de XML* (06-Ejemplos\Python).

Para el desarrollo en el lenguaje Python se aconseja usar la biblioteca:

```
import xml.etree.ElementTree as ET
```

esta biblioteca permite acceder al árbol DOM de los archivos XML y obtener las coordenadas de la etapa. También se puede utilizar la biblioteca **lxml**.

Para el manejo de esta biblioteca para leer el archivo **“etapaEsquema.xml”** y construir el árbol DOM en memoria, se deben consultar los ejercicios:

- a) Tema de teoría *“XPath”*, ejercicio **“02010-XPath.py”**. Ilustra el manejo de expresiones XPath, que **es obligatorio su uso para obtener los elementos y atributos del árbol DOM**.
- b) Tema de teoría *“Procesamiento y generación de XML”*, ejercicio **“Python/02000-XML.py”**. Ilustra el recorrido de un árbol DOM completo, tanto de archivos XML con prefijo y sin prefijo.

Para generar el código del archivo **“etapa.kml”** se puede hacer de **dos formas**;

1. Tema de teoría *“Procesamiento y generación de XML”*, ejercicio **“Nikon/Nikon-NMEA-KML.py”**

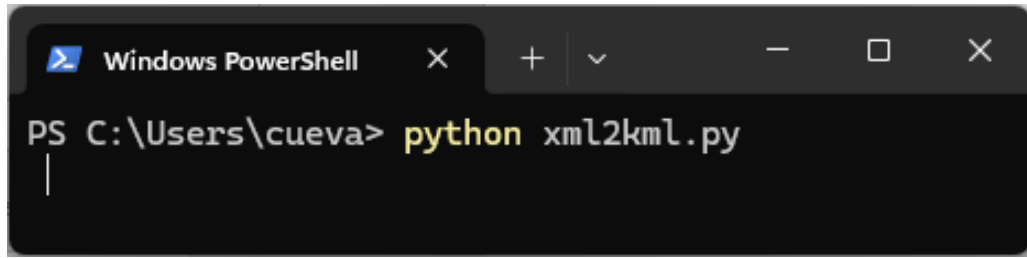
En este ejemplo se generan los archivos KML utilizando “generación de código basado en plantillas”. El programa se debe realizar siguiendo la siguiente organización:

- Leer el archivo **“etapaEsquema.xml”** y generar en memoria el árbol DOM
- Escribir el archivo **“etapa.kml”**
 - Escritura del **prólogo** con las etiquetas de encabezado del archivo KML
 - Escritura de las coordenadas de la etapa. Para extraerlas del árbol DOM se deben utilizar obligatoriamente **expresiones XPath**.
 - Escribir el **epílogo** con las etiquetas de cierre del archivo KML

2. Tema de teoría *“Procesamiento y generación de XML”*, ejercicio **“Python/02020-KML.py”**, en este ejemplo se usa una **clase** para generar archivos KML.

- Leer el archivo **“etapaEsquema.xml”** y generar en memoria el árbol DOM
- Extraer del árbol DOM mediante “expresiones XPath” los datos necesarios para generar el archivo KML.
- Escribir el archivo **“etapa.kml”**
 - Usar la clase Kml para generar las etiquetas de encabezado del archivo KML
 - Usar los métodos de la clase Kml para la escritura de las coordenadas de la etapa. Para extraerlas del árbol DOM se deben utilizar obligatoriamente **expresiones XPath**.
 - Usar los métodos de la clase Kml para las etiquetas de cierre del archivo KML

El programa **“xml2kml.py”** se ejecutará en línea de comandos en una “ventana cmd”, “Windows PowerShell”, “Símbolo del sistema” o “Terminal” usando el intérprete del lenguaje Python. Puede hacerse en los sistemas operativos Windows, Linux o MacOS.



```
Windows PowerShell
PS C:\Users\cueva> python xml2kml.py
```

Puede utilizarse “Google Earth” con la opción “**Archivo-> Abrir archivo KML local**” para visualizar el archivo KML con la imagen de la etapa. Si se utiliza el navegador Opera se puede capturar la pantalla en PDF. De esta forma se comprueba que el archivo KML generado se superpone correctamente con la etapa.

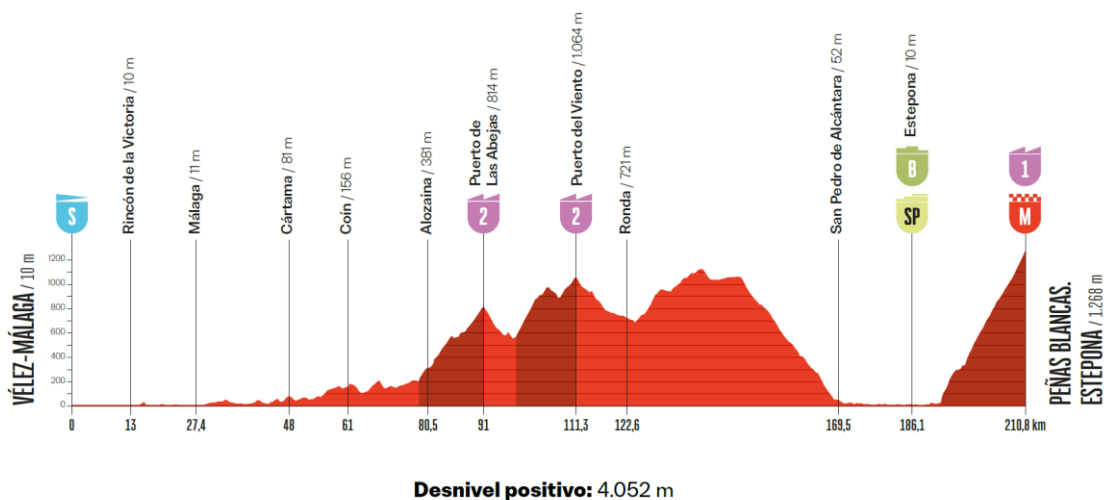
Ejercicio 3: Generar el archivo de altimetría de la etapa en formato SVG a partir del archivo etapaEsquema.xml usando el lenguaje Python

Se debe generar un archivo en formato SVG denominados “**altimetria.svg**” a partir del archivo “**etapaEsquema.xml**”. Debe usarse obligatoriamente el lenguaje Python. Se debe crear un archivo denominado “**xml2altimetria.py**”.

Es obligatorio utilizar “**expresiones XPath**” para obtener la información del archivo XML. También es obligatorio que el programa sea totalmente orientado a objetos (todas las funciones deben ser métodos de una clase).

El archivo SVG representa un perfil con la altimetría de la etapa y puede visualizarse con distintos agentes de usuario (navegadores). Se debe capturar la imagen de la altimetría de la etapa en un archivo en formato PDF denominado **altimetria.pdf**.

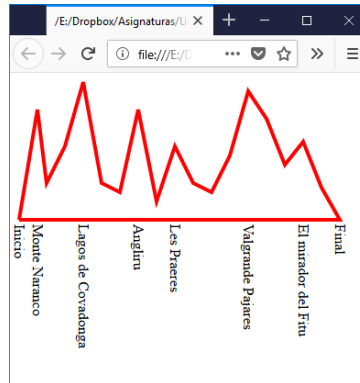
A continuación, se muestra la altimetría de la etapa 19 obtenida del libro de ruta oficial de “La Vuelta 2026”. Se debe obtener una imagen **similar** a la altimetría presentada en el libro de ruta.



Guía de la solución del ejercicio 3

El formato de los archivos SVG se explica en el *tema 1: Introducción a XML*. Un perfil altimétrico se puede construir con el elemento gráfico poli-línea (**"polyline"**), textos verticales y horizontales, así como líneas horizontales y verticales para mostrar la escala horizontal y vertical

A continuación, ejemplo de poli-línea con texto vertical (*"tema 1: Introducción a XML"*, archivo **polilinea-texto.svg**).



Para el desarrollo en el lenguaje Python se puede utilizar la biblioteca:

```
import xml.etree.ElementTree as ET
```

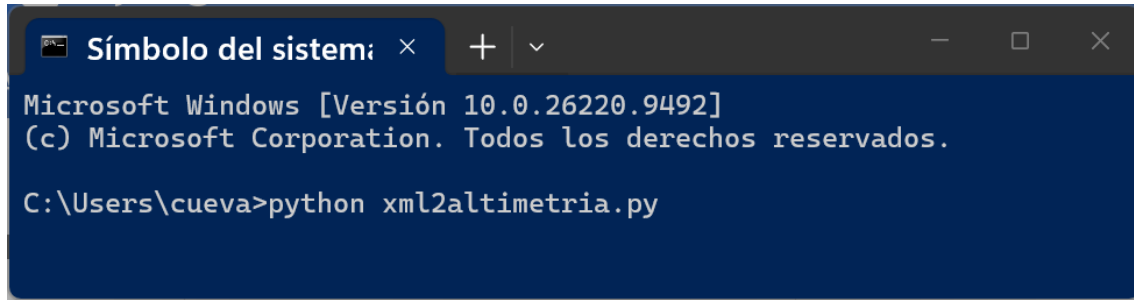
esta biblioteca permite acceder al árbol DOM de los archivos XML y obtener las distancias y altimetrías de la etapa por medio de expresiones XPath (obligatorio). También se puede utilizar la biblioteca **lxml**.

En el tema de teoría *"Procesamiento y generación de XML"*, ejercicio **"Python/02030-SVG.py"**, se usa una **clase** para generar archivos SVG. El procedimiento es el siguiente:

- Leer el archivo **"etapaEsquema.xml"** y generar en memoria el árbol DOM
- Extraer del árbol DOM mediante "expresiones XPath" los datos necesarios para generar el archivo SVG.
- Escribir el archivo **"altimetria.svg"**
 - Usar la clase **Svg** para generar las etiquetas de encabezamiento del archivo SVG
 - Usar los métodos de la clase **Svg** para la escritura de los datos de distancia y altitud de los puntos de la etapa. Para extraerlos del árbol DOM se deben utilizar obligatoriamente **expresiones XPath**.
 - Usar los métodos de la clase **Svg** para las etiquetas de cierre del archivo SVG

Para crear el archivo SVG se puede utilizar una polilínea siendo sus vértices los puntos de la etapa. Se puede cerrar la polilínea para hacer un efecto suelo en el perfil. También se puede rellenar.

El programa **"xml2altimetria.py"** se ejecutará en línea de comandos en una "ventana cmd", "Windows PowerShell", "Símbolo del sistema" o "Terminal" usando el intérprete del lenguaje Python. Puede hacerse en los sistemas operativos Windows, Linux o MacOS.



```
Símbolo del sistema: × + ▾  
Microsoft Windows [Versión 10.0.26220.9492]  
(c) Microsoft Corporation. Todos los derechos reservados.  
  
C:\Users\cueva>python xml2altimetria.py
```

Para convertir los archivos SVG a PDF se puede usar el navegador Opera y crear el archivo **“altimetría.pdf”**.

Resultados

En la carpeta XML se deben añadir los siguientes archivos a los que ya se tienen de la práctica anterior:

- etapaEsquema.xml
- etapa.xsd
- informeXMLesquema.pdf
- esquemaEtapa.svg
- esquemaEtapa.pdf

- etapa.kml
- xml2kml.py
- planimetria.pdf

- altimetria.svg
- xml2altimetria.py
- altimetria.pdf